

Take-Home Assignment: Short-Sell Surveillance

Overview

When a trading system sends a **sell** order to an exchange, the order must be correctly labelled as either a **long sell** (selling shares we already own) or a **short sell** (selling shares we do not own and must borrow). Regulators require this flag to be accurate, and an incorrectly flagged order can be a compliance breach.

Your task is to build a small surveillance tool. Given a day's stream of order events and a table of per-symbol reference data, replay the order activity and **print an alert** whenever a sell order's flag does not match the flag our rules say it should have had.

This mirrors a real surveillance job: read a feed of orders, keep track of our position as orders are placed and filled, and check each new sell against the rules.

Expected effort: 2-4 hours. We care more about correct, readable, well-tested logic than about polish or performance.

Inputs

You are given two CSV files.

- ref_data.csv - one row per symbol

Column	Meaning
symbol	instrument (stock/ETF/option/...) identifier
init_position	shares we hold in this symbol at the start of the day (can be 0)
Borrow	the maximum number of shares we are allowed to borrow (short) today

- orders.csv - one row per order **event**, in time order

Column	Meaning
event	event type (see below)
symbol	instrument (stock/ETF/option/...) identifier
client_order_id	our id for this order
orig_client_order_id	for amends/cancels: the id of the order being replaced or cancelled
side	Buy or Sell
qty	order quantity
leaves_qty	quantity still open after the event (populated on accepts/fills)
sell_flag	how the order was flagged: Long or Short

Event types:

- **New** - the order becomes a working (live) order. If it is a ****sell****, run the flagging rules on it, then add its quantity to ``working_long_qty`` or ``working_short_qty`` (depending on its flag).
- **NewAccept** - the exchange's acknowledgement that it received the new order. You can ignore it.
- **Amend** - an existing working order is replaced by a new version. ``orig_client_order_id`` is the old version and ``client_order_id`` is the new one; the quantity and/or flag may have changed. Remove the old version from the working totals, run the flagging rules on the new version if it is a sell, then add the new version to the working totals.
- **AmendConfirm** - the exchange's acknowledgement of an amend. You can ignore it.
- **Fill** - part or all of a working order executed. Lower that order's open quantity to ``leaves_qty`` (the executed amount is ``previous open quantity - leaves_qty``), which reduces the matching working total, and adjust ``net_position`` by the executed amount (a buy fill raises it, a sell fill lowers it). A fully filled order (``leaves_qty`` of 0) drops out of the working set.
- **Cancel / Cancelled** - the order is withdrawn; remove it from the working totals.

So only `New` and `Amend` sell orders are ever checked. Buys, fills, cancels and the acknowledgement events are never checked themselves, but – apart from the acknowledgements - they update the state you rely on when checking later sells.

States you may track

Process the events in timestamp order. For each symbol, you may need to maintain:

- **net_position** - start it at `init\_position`. Each **fill** changes it: a buy fill increases it by the filled quantity, a sell fill decreases it.
- **working_long_qty** - the total open quantity of all currently-working **sell** orders flagged `Long` for that symbol.
- **working_short_qty** - the total open quantity of all currently-working **sell** orders flagged `Short` for that symbol.

(A `New` sell adds itself to the working totals once it is live; an amend replaces the previous version; a fill reduces the order's open quantity; a full fill or a cancel removes it.)

The flagging rules

When a `New` or `Amend` **sell** order arrives, work out the flag it *should* have had, and alert if the actual `sell\_flag` is different. (Skip the order, if its symbol has no row in `ref\_data`.)

Everything turns on one idea: how many of the shares we own are still available to back a long sale - that is, our net position minus the shares already committed to other working long-sell orders.

Long sells (selling shares we own):

- A long sell is a promise to deliver shares we own, so the combined quantity of all our long sells must never exceed our net position - if they all filled, we must actually have the shares in hand.
- If the new order fits inside the shares we still own and haven't committed, then flagging it `Long` is correct.
- If we have no owned shares left to cover it (our owned shares are already fully committed to other long sells), the order is really a short sale. Flagging it `Long` is wrong - it should be `Short`. **Alert.**
- A single order must not sit on both sides of the line. If we still own *some* uncommitted shares but fewer than the order quantity, the order would be part long-covered and part short. For example, holding 100 shares and entering one 125-share long sell is invalid: it would have to be a 100-share long sell plus a 25-share short sell. A single order cannot cross the zero boundary like this. raise a 0-crossing **alert**.

Short sells (selling shares we do not own):

- A genuine short sale must be flagged `Short` in the first place. (A sell that should be short but was flagged `Long` is already caught by the long-sell rules above.)
- We are only permitted to short up to our `borrow` limit - the most shares we may borrow for the day.
- If a short order would take our total short position beyond the borrow limit - our combined working short quantity would exceed `borrow`, or our position after all sells would drop below `-borrow` - then it is an **uncovered short**. The expected flag is `Uncovered`. **Alert.**

Long when we actually hold the stock:

- Conversely, if we still hold plenty of shares so the order is fully covered by owned stock, but it was flagged `Short`, it should have been a long sale, expected flag `Long`. **Alert.**

Whenever the actual flag differs from the expected flag, print an alert that identifies the symbol, the flag that was used, the expected flag, and the figures behind the decision (net position, working long and short quantities, borrow).

Deliverable

A Python program that:

1. Reads `ref_data.csv` and `orders.csv` (assume they sit next to your script)
2. Replays the events in time order, maintaining the state described above.
3. Prints an alert line for every mis-flagged sell. Alerts may be emitted with plain `print()`.

You may use the standard library and/or `pandas` - whatever you prefer. There is no need for a database, web service, or any external system.

Along with the code, include a short `README` (a few sentences is fine) covering: how to run it and anything you'd add or change with more time.

Clearly state any assumptions you made, and why, in the README file itself. The rules above do not spell out every edge case on purpose - where something is ambiguous, decide what you think is most sensible, write down the assumption, and explain the reasoning behind it. We are at least as interested in how you reason about the gaps as in the code itself.

What we're looking for

- **Correctness** of the flag logic and the position/working-order bookkeeping, including the edge cases (amends that change quantity or flag, fills that reduce open quantity, acknowledgements that must be ignored, and missing reference data).
- **Clarity** - code we can read and reason about. Sensible names, small functions, a clean separation between "track state" and "decide the flag".
- **Testing** - even a handful of unit tests around the flag decision and the position tracking tell us a lot. We don't expect exhaustive coverage.
- We are **not** grading on performance, fancy output formatting, or an elaborate CLI.

Using AI tools

You are welcome to use AI assistants while working on this. What we ask is that the submission is genuinely **yours**, not a deliverable generated wholesale by a tool. Use AI the way you would in real work - to speed things up or check your thinking - but make sure you fully understand every line of code and the logic behind it. In the follow-up discussion we will walk through your solution and ask why you made particular choices and how it would behave in cases we suggest, so you should be able to explain and defend all of it without help.

Questions

If anything is unclear, please ask - sensible questions are welcome and won't count against you. Email careers@oracson.com and we'll get back to you.

Good luck!

A few worked examples:

These illustrate the rules; they are not the full set of cases in the data.

- We hold 100 shares and have no other working sells. A single 125-share sell flagged `Long` is invalid: 100 shares are covered by stock we own and 25 are not, and one order cannot be both. → Alert.
- We own 0 shares of a symbol (or have already committed all our owned shares to other long sells). A further sell flagged `Long` is really a short sale → expected `Short`. → Alert.
- We hold 800 shares and place a sell flagged `Short`. We own far more than enough to cover it, so this is a long sale → expected `Long`. → Alert.
- Our working short sells already total 2,000 shares against a `borrow` limit of 0. Another short sell arrives, pushing our short past what we can borrow → Uncovered. → Alert.
- A sell that was flagged `Long` when it should have been short is later amended to `Short` while we are net short and within borrow. After the amend it matches the expected flag → no alert for the amended version.